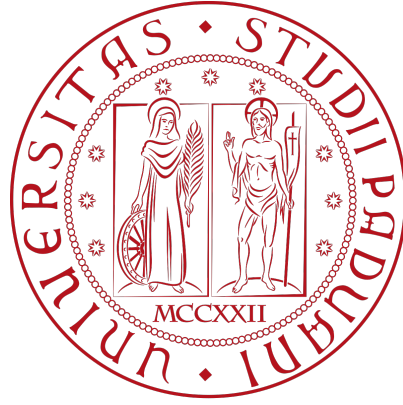




UNIVERSITÀ DEGLI STUDI DI PADOVA
Department of Mathematics “Tullio Levi-Civita”
Master’s Degree in Computer Science
Academic year 2025/2026

CPS and IoT Security

A lightweight distributed anomaly-based
Intrusion Detection System for
RPL networks in Contiki-NG and Cooja

Author: **Riccardo Fabbian**

Student ID: **2140711**

Period of work: **September 2026**

Contents

1	Introduction	4
1.1	The Reference Paper	4
1.2	Objective	4
1.3	Reproducibility	5
2	Background	6
2.1	The Protocol Stack	6
2.2	How RPL Builds a Network	6
2.3	Trickle: Why the Network Is Normally Quiet	6
2.4	Anomaly-Based Intrusion Detection	7
3	Threat Model	8
3.1	The Neighbour Attack	8
3.2	The DIS Attack	8
3.3	Out of Scope	8
4	The Reference Model	9
5	System Design	11
5.1	Node Roles	11
5.2	Detector State	11
5.3	Integer Arithmetic	11
5.4	Response	11
6	Implementation	13
6.1	Toolchain	13
6.2	Instrumentation	13
6.3	Attackers	13
6.4	Logging and Ground Truth	14
6.5	Verification	14
7	Experimental Methodology	15
7.1	Metrics and the Unit of Measurement	15
8	Results	17
8.1	Baseline Behaviour	17
8.2	The DIS Attack	17
8.3	The Neighbour Attack	18
8.4	Two Structural Findings	19

8.4.1	Blind Spots at Certain Neighbourhood Sizes	19
8.4.2	Multiple Attackers Mask One Another	20
8.5	Fixed Versus Sliding Window	21
8.6	Network Impact	23
8.7	Sensitivity to the DIS Threshold	24
8.8	Sensitivity to the Attack Rate	25
8.9	Resource Overhead	26
9	Discussion	27
10	Limitations and Future Work	28
11	Conclusion	29

List of Figures

1	Topology of the 40-node scenario with 30% attackers. The root is at the top centre; the dashed circle is the 50 m transmission range of one attacker	15
2	DIO messages received per neighbour over successive observation windows. The attacker is highlighted; honest neighbours settle to 0–1 message per window once Trickle converges	17
3	DIS messages received per neighbour. Honest nodes stop after joining; the attacker’s stream is separated from them by a wide margin	18
4	Dynamic threshold, neighbourhood mean, and the attacker’s DIO count at a detecting monitor. The attacker stays above the threshold once the attack begins	18
5	Alerts over the course of one run. Filled markers are true positives on the attacker, hollow markers are false positives caused by integer quantisation of small counts	19
6	True and False Positive Rates as a function of network size, for both attacks	22
7	Fixed versus sliding observation window: detection rate, false-positive rate, and mean detection latency	22
8	Packet delivery ratio and control-plane overhead, attack-free runs against attacks, for both detector modes	24
9	Detection latency and True Positive Rate as a function of the attack period. The sliding mode is faster at every rate but loses the slow DIS attacker entirely	25

1 Introduction

The Internet of Things connects a very large number of small devices that run on batteries, carry a few kilobytes of memory, and talk to each other over short-range radio. Networks of this kind are called Low-Power and Lossy Networks, and they cannot use the routing protocols of the ordinary internet: those protocols assume plentiful memory, stable links, and a power socket. For this reason the IETF standardised a dedicated routing protocol for them, RPL, described in RFC 6550.

“In RPL, encryption and authentication methods can only protect networks from external attack and cannot protect networks against internal attack.”

This sentence, taken from the reference paper of this project, is the reason the work exists. A device that has legitimately joined an RPL network already holds the network key. Everything it sends is valid, correctly encrypted, and impossible to distinguish from honest traffic by cryptographic means alone. If such a device decides to misbehave, the only remaining defence is to watch how it behaves and notice that its behaviour is abnormal. That is the job of an Intrusion Detection System, and it is what this project builds, reproduces, and evaluates.

1.1 The Reference Paper

The project reproduces the model proposed by B. Farzaneh, M. A. Montazeri and S. Jamali in *An Anomaly-Based IDS for Detecting Attacks in RPL-Based Internet of Things*, presented at the 5th International Conference on Web Research (ICWR) in 2019.

The paper proposes a detection method for two internal attacks specific to RPL, the neighbour attack and the DIS attack. Its main quality is that it is genuinely lightweight: the detector is a handful of counters and one threshold formula, it runs on every ordinary node, and it needs no central monitor and no reporting traffic. The authors report a True Positive Rate of 100% for the DIS attack with a False Positive Rate of 0%, and between 94% and 100% for the neighbour attack, on networks of 20, 30 and 40 nodes.

1.2 Objective

The work has two parts. The first is a reproduction: implement the paper’s detector and its two attacks on a current software stack, run the same experimental matrix, and check whether the reported results still hold. The second is an original contribution: the paper evaluates its counters once every five minutes, which means an attack

starting just after an evaluation goes unnoticed for almost the whole window. This project therefore adds a second detector mode that uses a shorter, sliding observation window, and measures what is gained and what is lost.

The reproduction is not a straight port. The paper used Contiki 2.7, a version now more than a decade old; this project uses Contiki-NG v5.2, in which the MAC layer used by the authors no longer exists and the routing layer has been rewritten. Every difference that this forced is documented and, where it affects the results, discussed.

1.3 Reproducibility

The whole environment is pinned inside a Docker image: a fixed Contiki-NG commit, a fixed compiler, and a prebuilt simulator. Every scenario is generated from a random seed by a script rather than edited by hand, and the simulator is deterministic for a given seed. Running the same scenario twice produced byte-identical logs, so any variation reported in this document comes from changing the seed and not from run-to-run noise. All figures and tables in the results section are regenerated from the raw logs by the analysis scripts.

2 Background

This section introduces the protocol machinery that the attacks abuse and the detector observes. A reader familiar with RPL and Trickle can move directly to Section 3.

2.1 The Protocol Stack

A Low-Power and Lossy Network places IPv6 over an adaptation layer called 6LoW-PAN, which compresses IPv6 headers so that a packet fits inside the 127-byte frame of IEEE 802.15.4, the radio standard used by these devices. Above this sits RPL, and above RPL the application, which in this project is a simple UDP sensor report.

2.2 How RPL Builds a Network

RPL organises the network as a tree, formally a Destination-Oriented Directed Acyclic Graph or DODAG, rooted at one node called the root. The root is normally a border router: a slightly more capable device with a real connection to the internet, which is also the destination of all the data the network produces.

The tree is built from the root outwards. The root announces itself; the nodes that hear the announcement learn that they are one hop away and announce themselves in turn; their neighbours learn that they are two hops away, and so on. Each node stores a **rank**, which expresses its distance from the root, and selects one neighbour with a lower rank as its **preferred parent**. Every data packet is then simply handed to the preferred parent, and by induction it reaches the root.

Three control messages maintain this structure:

- **DIO** (DODAG Information Object): the announcement described above. A node broadcasts a DIO to say who it is, which DODAG it belongs to, and what its rank is. This is the message the detector counts.
- **DIS** (DODAG Information Solicitation): a request for a DIO. A node that has just been switched on broadcasts a DIS to ask its neighbours to announce themselves, so that it can join without waiting. After joining, a well-behaved node has almost no reason to send another one.
- **DAO** (Destination Advertisement Object): sent upwards to inform the root of a node's existence, so that the root can also send packets down the tree.

2.3 Trickle: Why the Network Is Normally Quiet

If every node broadcast a DIO at a fixed rate, the batteries would be exhausted in days. RPL therefore paces DIOs with an algorithm called **Trickle**. The rule is simple: when something changes, broadcast often; while nothing changes, double

the interval each time, up to a configured maximum. A network that has just been switched on is noisy for a minute or two; a converged network is almost silent, with each node broadcasting once every several minutes.

Trickle has a second rule that matters a great deal here: *receiving a DIS resets the timer back to its minimum*. A single solicitation therefore makes every node that hears it noisy again.

These two rules together are the whole basis of the detector and of both attacks. Because a converged network is quiet, a node that broadcasts many DIOs in a five-minute window is statistically obvious. Because a DIS resets everyone's timer, a node that broadcasts many DIS messages can make an entire neighbourhood expensive to run without sending much itself.

2.4 Anomaly-Based Intrusion Detection

Intrusion detection systems are usually divided into signature-based systems, which match traffic against known attack patterns, and anomaly-based systems, which build a profile of normal behaviour and flag whatever deviates from it. Signature-based detection is precise but blind to anything not already catalogued; anomaly-based detection can catch new attacks but must accept a rate of false alarms.

The reference paper is anomaly-based. Its profile is the number of DIO messages a node receives from each of its neighbours in a fixed time window, and its claim is that these counts follow a normal distribution across the neighbourhood, so that a threshold expressed in standard deviations separates honest neighbours from attackers.

A further distinction concerns where the detector runs. It may be centralised on the border router, which is simple but requires every node to report what it hears, or distributed across the nodes themselves. The paper's design is fully distributed and **stand-alone**: each node monitors its own neighbours and decides alone, exchanging no detection traffic whatsoever. This is its main advantage over earlier systems such as SVELTE, which concentrate the analysis on the border router.

3 Threat Model

The network consists of static nodes with constrained memory and energy, one root that also acts as the data sink, and normal nodes that forward traffic towards it. The attacker is an **internal** node: it has joined the DODAG legitimately, it holds valid keys, and every message it sends is well formed. It is not assumed to break link-layer encryption; the attacks exploit protocol behaviour after joining, which is precisely the case that cryptography cannot address.

Every normal node runs the detector and observes only what its own radio can hear. There is no cooperation, no shared blacklist, and no reporting to the root.

3.1 The Neighbour Attack

In the neighbour attack the malicious node broadcasts DIO messages far more often than the protocol requires. Each of these messages forces every node in range to process it and reconsider its parent selection. A node may add the attacker to its parent set, or even select it as preferred parent, on the basis of an announcement that does not reflect a usable route; the paper notes that the attacker may not even be within range of the intended destination, so the path degrades. The cost to the victims is processing, radio time, and routing instability.

3.2 The DIS Attack

In the DIS attack the malicious node broadcasts DIS messages instead. Because a received DIS resets the Trickle timer, every neighbour returns to its minimum broadcast interval and starts emitting DIOs frequently again. The attacker therefore amplifies its own effort: a modest stream of solicitations produces a large volume of traffic from honest nodes. This is the reason the DIS attack turns out, in Section 8.6, to be by far the more expensive of the two for the network.

3.3 Out of Scope

Following the reference paper, the wormhole, sinkhole, selective-forwarding, rank and version-number attacks are not implemented. They would broaden the work beyond the paper's contribution and make it harder to attribute any observed effect to the detector under study.

4 The Reference Model

The paper describes its detector as four phases repeated in every observation window. The description below follows the original notation.

Phase I. Each node counts, for every neighbour, the number of DIO messages received during the current time window.

Phase II. At the end of the window the node computes the mean μ and the standard deviation σ of these per-neighbour counts, and derives a threshold

$$T = \mu + k\sigma \quad (1)$$

where k is a coefficient that depends on how many neighbours the node has. The authors obtained k empirically: they simulated many attack-free networks of different sizes and densities, recorded for each neighbourhood size the largest deviation that occurred naturally, and fitted a curve through those maxima. The result is a fourth-degree polynomial in the number of neighbours x :

$$k = -5 \cdot 10^{-5}x^4 + 0.0037x^3 - 0.0899x^2 + 0.9281x - 0.7903 \quad (2)$$

The logic is that a node with few neighbours needs a small margin, while a node with many neighbours sees more natural variation and needs a larger one.

Phase III. Any neighbour whose DIO count exceeds T is declared a neighbour attacker. Independently, any neighbour whose DIS count exceeds a fixed threshold is declared a DIS attacker. The authors measured the maximum number of DIS messages a normal node ever produced across all their scenarios, found it to be 3, and adopted that value as the threshold.

Phase IV. A node identified as an attacker is **temporarily blocked**. The reason for making the block temporary is explicitly stated in the paper: an honest node may occasionally be flagged by accident, and a permanent exclusion would be an excessive penalty. Only after a counter of repeated detections reaches a threshold does the block become permanent. In the paper's simulations the block counter threshold is 2 and the temporary block lasts one minute.

Table 1 lists the parameters of the reference simulations, which this project adopts as its baseline.

Parameter	Value in the reference paper
Operating system	Contiki 2.7
Simulator	Cooja
Mote	Tmote Sky (10 kB RAM, 48 kB ROM)
Routing	ContikiRPL
MAC / duty cycle	CSMA/CA over ContikiMAC
Radio medium	Unit Disk Graph Medium, distance loss
Range	50 m transmission, 100 m interference
Deployment area	100 m $\times$ 100 m
Number of nodes	20, 30, 40
Attackers	1 node, 20%, 30%
Traffic	Constant bit rate, UDP
Simulation duration	30 minutes
Attack start	75 s after the start of the simulation
Observation window	5 minutes
DIS threshold	3
Block threshold	2, temporary block of 1 minute

Table 1: Simulation parameters of the reference paper

5 System Design

5.1 Node Roles

A simulation contains three kinds of device. The **root** starts the DODAG and receives the UDP traffic; it deliberately runs no detector, because the paper’s system is distributed and keeping the root passive proves that no detection result depends on it. A **normal node** joins the network, sends one UDP datagram to the root every 30 seconds, and runs the detector on everything it hears. An **attacker** behaves exactly like a normal node, including the UDP traffic that makes it look operational, but additionally injects control messages on a timer; it runs no detector either, since an attacker has no reason to police its neighbours.

5.2 Detector State

Each normal node keeps one table with a row per neighbour it has ever heard from. A row holds the neighbour’s identifier, its DIO and DIS counters for the current window, and the response state: how many times it has been detected so far, whether it is permanently blocked, and, if it is temporarily blocked, the time at which the block expires.

This table is deliberately **independent of the routing layer’s own neighbour table**. RPL only tracks neighbours it might use as parents, whereas the paper’s detector must count messages from every node it can hear. Both counts are logged so that the difference can be examined; the paper does not specify which of the two its “neighbour list” refers to, and this is recorded as an interpretation.

5.3 Integer Arithmetic

The target device has no floating-point unit, so evaluating the threshold formula naively would be both slow and large. The implementation therefore works entirely in integers scaled by a factor of 1000. The polynomial for k is not evaluated at run time at all: since the number of neighbours is a small integer, the values of k are precomputed on the host for every neighbourhood size from 0 to 40, scaled by 1000, and compiled in as a lookup table of 41 entries. The standard deviation uses an exact integer square root. The consequence is that the detector performs no division or multiplication wider than 64 bits and contains no floating-point code whatsoever.

5.4 Response

When a neighbour is flagged, the node stops accepting its DIO and DIS messages: they are discarded before the routing layer processes them. The effect is exactly what

the attack needs to be neutralised, since a discarded DIS cannot reset the Trickle timer and a discarded DIO cannot make the attacker a parent.

This choice deserves justification. A more direct implementation would delete the attacker from the routing layer’s neighbour table, but doing so from outside the routing code risks corrupting its internal state, and the routing layer would simply re-learn the neighbour from its next message. Filtering the message before it is parsed achieves the intended effect without touching any routing structure.

One detail is essential for the paper’s escalation to work: **counting continues while a neighbour is blocked**. If the node stopped counting, a blocked attacker would appear silent, would never be detected again, and would never reach the permanent block. Counting through the block is what allows repeated detection to escalate.

6 Implementation

6.1 Toolchain

The project targets Contiki-NG release v5.2 and the Cooja simulator, both pinned by commit inside a Docker image built on the official Contiki-NG toolchain image. Cooja runs headless, driven by a control script embedded in each scenario file, which is what makes it possible to execute a matrix of one hundred and forty-eight simulations without human interaction. The image records the exact versions of Contiki-NG, Cooja, Java and the two cross-compilers in a file so that they can be quoted in this report.

Simulations run on the `cooja` mote, a native build of the firmware executed directly by the simulator. This is orders of magnitude faster than emulating the Tmote Sky instruction by instruction: thirty simulated minutes of a forty-node network complete in a few seconds. The Tmote Sky target is still used, but only for the memory measurements of Section 8.9.

6.2 Instrumentation

The detector must see every DIO and DIS the radio receives. Contiki-NG registers exactly one handler per ICMPv6 message type, so a second handler cannot simply be added from application code. The solution is a patch of two lines against the routing layer, applied automatically when the Docker image is built:

- at the top of the DIS input function, and immediately after the sender address becomes available in the DIO input function, a call is inserted to a hook function provided by the detector;
- the hook returns a value that decides whether the message continues to be processed or is discarded.

This single hook therefore serves both purposes described in the design: it is where messages are counted, and it is where blocked neighbours are filtered. The patch is guarded by a preprocessor symbol, so with the feature disabled the routing layer is byte-for-byte the original. Parent changes are captured through a callback that the routing layer already exposes, and required no patch.

6.3 Attackers

The two attacker firmwares reuse the routing layer's own DIO and DIS transmission functions, called on a timer between a configurable start time and a configurable end time. Using the protocol's own functions rather than crafting packets guarantees

that the injected messages are indistinguishable from legitimate ones, which is the point of the threat model. The transmission rate is a build parameter, so the same source produces a moderate or an aggressive attacker; the paper’s random interval of 5 to 60 seconds for the DIS attack is available as a special value and is the default.

6.4 Logging and Ground Truth

Nodes emit tab-separated records over the serial line: one line per received control message, one line per neighbour at every evaluation, one line for the computed profile and threshold, one line per alert, one line per blocking action, and periodic status lines with rank, parent and traffic counters. The simulator’s control script prefixes every line with the simulated time and the node identifier, so the entire run is a single machine-readable log.

Attackers additionally log every message they inject, and the scenario generator writes a separate ground-truth file listing which nodes are attackers, of which type, and during which interval. The analysis joins alerts against this file. Ground truth is never inferred from the detector’s own output, which would make the evaluation circular.

6.5 Verification

The detector’s arithmetic is duplicated in Python as a bit-exact reference model. A test replays every stored simulation log, recomputes the profile and the alerts from the logged per-neighbour counts, and requires the results to match the firmware exactly, including the intermediate mean, standard deviation, coefficient and threshold. The test suite contains 28 tests in total and also covers the edge cases prescribed in the project plan: zero neighbours, one neighbour, identical counts, a single large outlier, and DIS counts exactly at and one above the threshold.

7 Experimental Methodology

Nodes are placed on a regular grid inside the paper’s 100 m $\times$ 100 m area, with the root at the top centre and a spacing of 20 metres, reproducing the layout of the reference paper. Figure 1 shows the largest configuration. The radio model is the Unit Disk Graph Medium with a transmission range of 50 metres and an interference range of 100 metres.

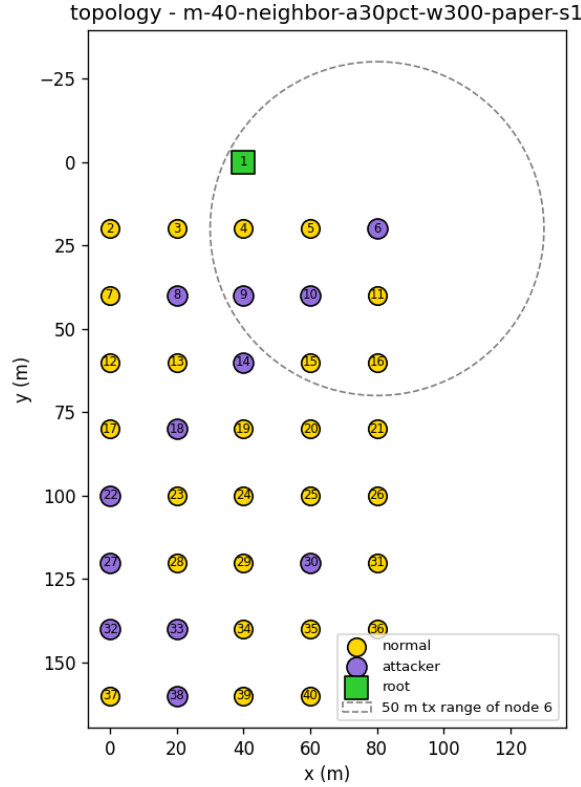


Figure 1: Topology of the 40-node scenario with 30% attackers. The root is at the top centre; the dashed circle is the 50 m transmission range of one attacker

Every node sends one UDP datagram to the root every 30 seconds. Attacks begin 75 seconds after the start of the simulation, as in the paper, and last 10 minutes; simulations run for 30 minutes. The main matrix covers 20, 30 and 40 nodes with one attacker, 20% and 30% attackers, for both attack types and both detector modes, repeated over 5 independent seeds. Two additional sweeps, on the DIS threshold and on the attack rate, use 3 seeds. Development used smaller 10-node, 5-minute scenarios so that the iteration cycle stayed short.

7.1 Metrics and the Unit of Measurement

Defining what counts as one sample is the least obvious methodological decision in this work, and it changes the numbers considerably.

The **node-level** view classifies each node once per run: an attacker is a true positive if at least one monitor flagged it with the rule appropriate to its attack, and a normal node is a false positive if at least one monitor flagged it with that same rule. This is the view that corresponds to the paper’s results table, and it is the one reported throughout Section 8. It also reflects what an operator would experience, since one correct detection anywhere in the neighbourhood is enough to act.

The **decision-level** view instead treats every single judgement, by one monitor about one neighbour at the end of one window, as an independent sample. It is a stricter measure and it is the one that exposes the structural weakness described in Section 8.4, because it shows that many monitors fail to flag an attacker that one of their peers catches easily.

Both are computed for every run. Detection latency is measured from the start of the attack to the first correct alert.

8 Results

8.1 Baseline Behaviour

In attack-free runs the network converges quickly and behaves as expected: every node obtains a rank and a parent within about 60 seconds, every UDP datagram reaches the root, and nodes change parent on average once, during the initial convergence.

Figure 2 shows the number of DIO messages received per neighbour over successive windows. The shape is exactly the one Trickle predicts: a burst while the network converges, then a decline to between zero and one message per neighbour per window. This quiet steady state is what makes the detector possible. The per-neighbour DIS count never exceeded 2 in any attack-free run, which independently confirms the paper’s choice of 3 as the DIS threshold.

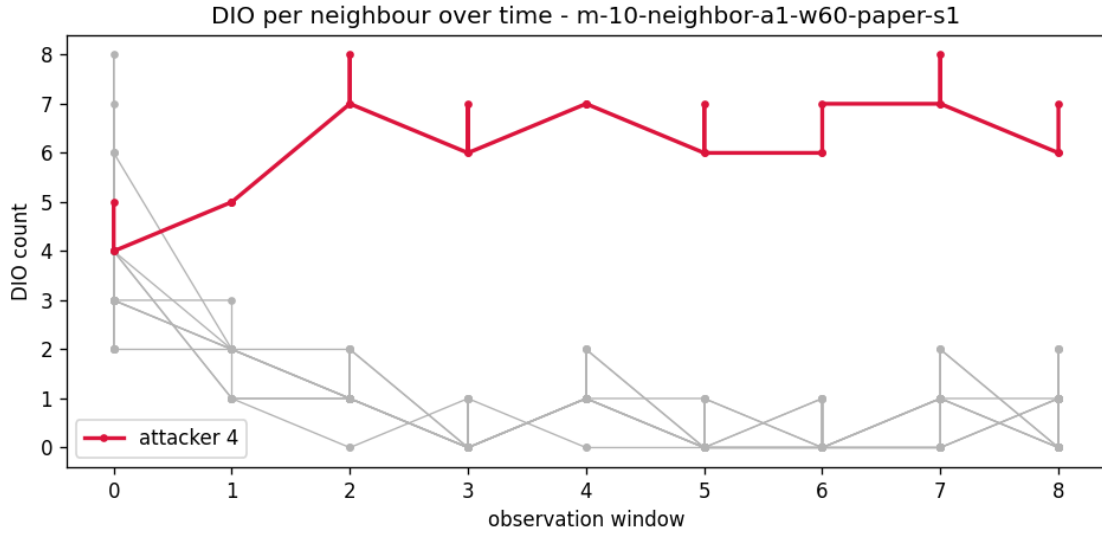


Figure 2: DIO messages received per neighbour over successive observation windows. The attacker is highlighted; honest neighbours settle to 0–1 message per window once Trickle converges

8.2 The DIS Attack

The DIS attack reproduces the reference paper exactly. At 20, 30 and 40 nodes, and with one attacker, 20% and 30% attackers, the node-level True Positive Rate is 100% and the False Positive Rate is 0%, in every seed and in both detector modes.

The reason the result is so clean is visible in Figure 3. Honest nodes send at most two DIS messages in their whole lifetime, immediately after being switched on, while the attacker produces a continuous stream. The two populations do not overlap at all, so a fixed threshold placed between them cannot make a mistake in either

direction. This is the ideal case for a threshold-based detector, and it explains why the paper’s DIS results are perfect rather than merely good.

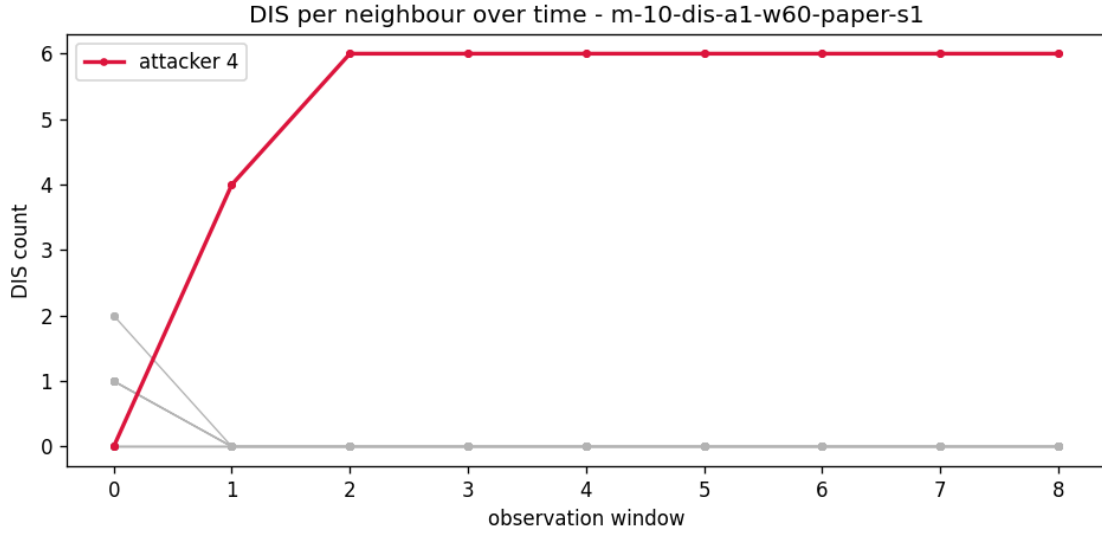


Figure 3: DIS messages received per neighbour. Honest nodes stop after joining; the attacker’s stream is separated from them by a wide margin

8.3 The Neighbour Attack

A single neighbour attacker is also detected reliably: the node-level True Positive Rate is 100% at 20 nodes, across all seeds. Figure 4 shows the mechanism at the node that detects it, with the attacker’s count crossing above the dynamic threshold while the neighbourhood mean stays low.

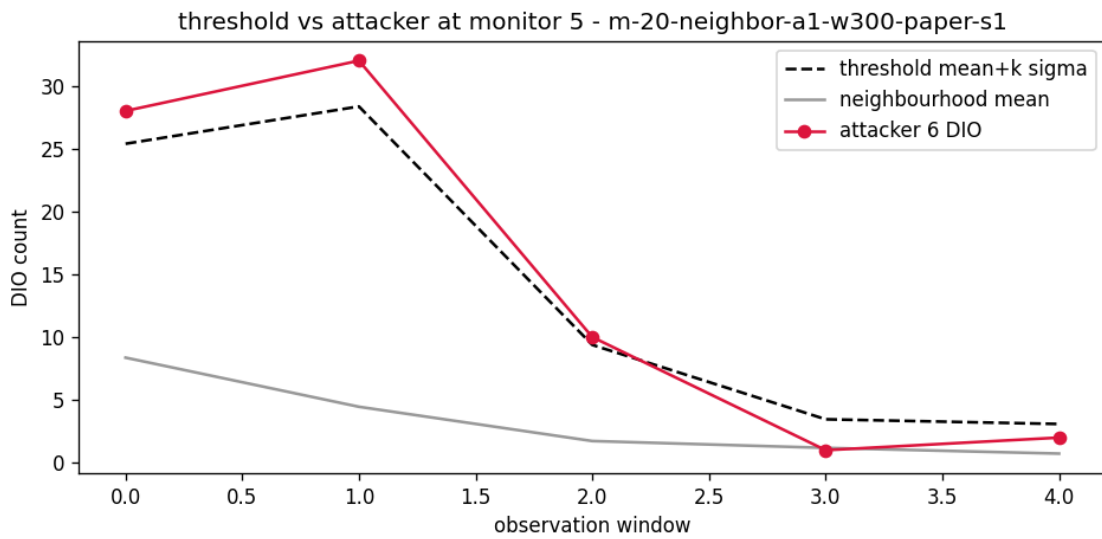


Figure 4: Dynamic threshold, neighbourhood mean, and the attacker’s DIO count at a detecting monitor. The attacker stays above the threshold once the attack begins

Figure 5 places every alert of one run on a timeline. Filled markers are correct detections of the attacker; hollow markers are false positives on honest nodes. The false positives are not random noise: they are cases where an honest neighbour happened to send two or three DIOs in a window in which the others sent zero or one. Because the counts are small integers, such a neighbour can be several standard deviations above a mean that is itself below one, and the threshold formula flags it. This quantisation effect is the entire source of the false-positive rate on the neighbour rule, and it accounts for the few per cent measured here as well as, most probably, for the 0.2% to 9.6% reported in the paper.

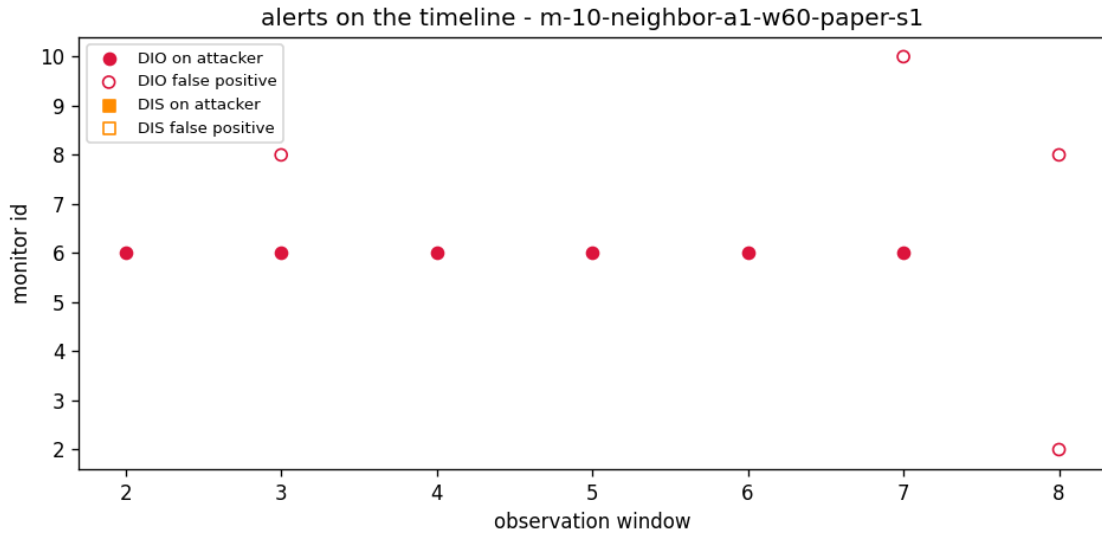


Figure 5: Alerts over the course of one run. Filled markers are true positives on the attacker, hollow markers are false positives caused by integer quantisation of small counts

However, the True Positive Rate of the neighbour attack **collapses as the proportion of attackers grows**: from 100% with a single attacker to 33% with 20% attackers and to 5% with 30% attackers. The paper reports between 94% and 99% in the same configurations. This discrepancy is not a defect of the implementation but a property of the threshold itself, and the next section explains it.

8.4 Two Structural Findings

8.4.1 Blind Spots at Certain Neighbourhood Sizes

Algorithm 1 of the paper computes the standard deviation by dividing by the number of neighbours, which makes it the *population* standard deviation, and it computes it over a set that *includes the attacker*. This has a consequence the authors do not discuss.

Samuelson’s inequality states that in a sample of n values, no single value can be

further than $\sqrt{n-1}$ standard deviations from the mean. A lone attacker is therefore detectable only if

$$\sqrt{n-1} > k(n) \quad (3)$$

and this is a hard limit: it holds no matter how many messages the attacker sends, because every extra message raises the mean and the standard deviation that define its own threshold.

The difficulty is that the paper obtained k by fitting the largest deviations observed in normal traffic, and those maxima are themselves constrained by the same inequality. The fitted curve consequently runs very close to the bound and crosses it twice. Evaluating both sides for every neighbourhood size gives the blind spots in Table 2: a node with 5, 6 or 7 neighbours, or with 28 to 32 neighbours, can never flag a lone attacker.

Neighbours n	Relation between $\sqrt{n-1}$ and $k(n)$	Lone attacker
2–4	$\sqrt{n-1}$ above $k(n)$, narrowly at $n = 4$	detectable
5–7	$k(n)$ above $\sqrt{n-1}$	never detected
8–27	$\sqrt{n-1}$ above $k(n)$	detectable
28–32	$k(n)$ above $\sqrt{n-1}$, polynomial peak	never detected
33–40	$\sqrt{n-1}$ above $k(n)$	detectable

Table 2: Neighbourhood sizes at which the published threshold cannot detect a single attacker, at any attack rate

The prediction was verified experimentally. In one scenario an attacker sending 13 to 14 DIOs per window, against a normal rate of about one, was heard by exactly three monitors. The monitor with 8 neighbours flagged it in all seven windows of the attack. The monitors with 5 and 6 neighbours never flagged it, in any window, exactly as the inequality requires.

The distributed design is what saves the system: a single monitor outside the blind range is sufficient for the attacker to be caught, which is why the node-level True Positive Rate remains 100% even though most individual monitors fail.

8.4.2 Multiple Attackers Mask One Another

The second finding follows from the same statistics. When several attackers are within range of the same monitor, they all contribute their inflated counts to the mean

and to the standard deviation. The threshold rises with them, and no individual attacker stands out from the group. The effect is the generalisation of the previous inequality from one outlier to several, and it is what drives the collapse of the True Positive Rate at 20% and 30% attackers.

The measured drop is steeper than the paper’s. Two differences plausibly account for the gap. The attackers here are placed by seed and frequently fall within radio range of one another, so they share monitors; and the attacker implemented here floods at a constant rate, whereas the paper’s attacker rebroadcasts the DIOs it receives, which produces a different and less synchronised traffic pattern. Neither difference changes the mechanism, and the conclusion stands as a limitation of the published threshold when attackers collude.

8.5 Fixed Versus Sliding Window

The original contribution of this project is a second detector mode. Both modes use identical counters, identical thresholds, identical topologies and identical seeds; the only difference is how often the counters are examined. The paper’s mode evaluates once every 300 seconds. The sliding mode keeps the last 60 seconds as six buckets of 10 seconds each and re-evaluates the rolling total every 10 seconds.

Table 3 reports the full matrix, and Figure 6 shows the same data as a function of network size.

Nodes	Attack	Att.	Mode	TPR %	FPR %	Lat. s	DIO/min
20	DIS	1	paper	100.0	0.0	225	27.4
20	DIS	1	sliding	100.0	0.0	45	7.1
30	DIS	20%	paper	100.0	0.0	225	41.9
30	DIS	20%	sliding	100.0	0.0	45	15.3
40	DIS	30%	paper	100.0	0.0	225	61.0
40	DIS	30%	sliding	100.0	0.0	15	23.6
20	neighbour	1	paper	100.0	7.4	225	6.2
20	neighbour	1	sliding	100.0	52.6	29	5.4
30	neighbour	20%	paper	33.3	1.7	225	11.5
30	neighbour	20%	sliding	46.7	12.5	25	9.8
40	neighbour	30%	paper	5.0	0.0	225	24.3
40	neighbour	30%	sliding	13.3	1.4	11	20.5

Table 3: Main matrix, node-level metrics, mean over 5 seeds. DIO/min is the number of DIO messages received per node per minute; the attack-free baseline is 4.6

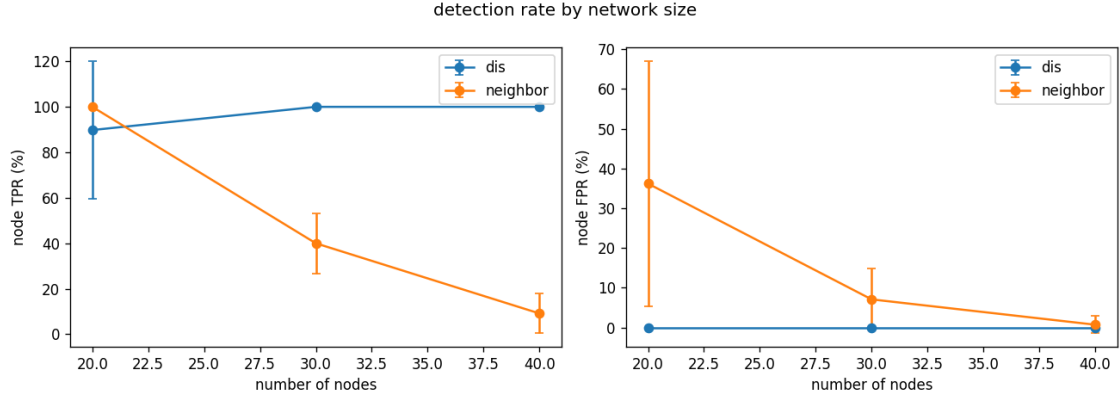


Figure 6: True and False Positive Rates as a function of network size, for both attacks

Averaged over the six attack configurations, the sliding window reduces the mean detection latency from 225 seconds to 28 seconds, an improvement of roughly an order of magnitude, while raising the mean node-level False Positive Rate from 1.5% to 11.1%. Figure 7 summarises the comparison.

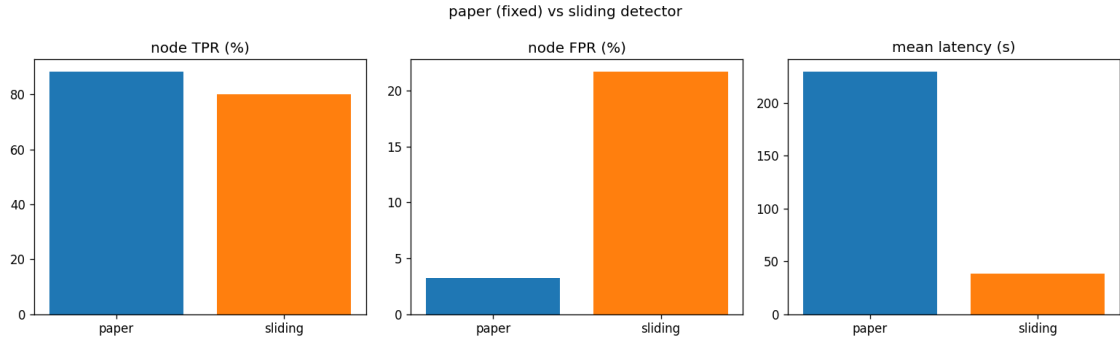


Figure 7: Fixed versus sliding observation window: detection rate, false-positive rate, and mean detection latency

Two results were not anticipated. The first is that the sliding window *improves* the True Positive Rate under multiple attackers, from 33% to 47% and from 5% to 13%: a shorter window captures an attacker’s burst before the counts of the other attackers have accumulated into the mean, so it partially resists the masking effect. The second is that it substantially reduces the traffic caused by the DIS attack, from 61 to 24 received DIOs per node per minute in the largest configuration, because the attackers are blocked much earlier and therefore reset far fewer Trickle timers. A faster detector is not only more responsive; it is also cheaper for the network it protects.

The costs are equally concrete. In attack-free runs the node-level False Positive Rate reaches 100% in sliding mode against 41% in the paper mode, simply because

evaluating thirty times more often gives every honest node thirty times more opportunities to be flagged once by the quantisation effect. More seriously, the sliding mode issues approximately one hundred *permanent* blocks per run **even when no attacker is present**, against none in the paper mode. The reason is that the response parameters, two temporary blocks before a permanent one, were calibrated for one decision every five minutes and not for thirty. A sliding detector cannot inherit them unchanged: it needs a higher block counter threshold, or a requirement that detections be consecutive, before the response is safe to deploy.

8.6 Network Impact

Nodes	Attack	Attackers	PDR	Delay ms	DIO/min
20	baseline	–	1.000	38	4.6
20	neighbour	1	1.000	38	6.2
20	DIS	1	1.000	38	27.4
30	neighbour	20%	1.000	50	11.5
30	DIS	20%	1.000	51	41.9
40	neighbour	30%	1.000	63	24.3
40	DIS	30%	1.000	64	61.0

Table 4: Network impact of the attacks, paper mode. Packet Delivery Ratio and delay are unaffected; the cost falls entirely on the control plane

Table 4 and Figure 8 report the effect of the attacks on the network itself. Packet delivery stays at 1.000 and the end-to-end delay is unchanged, which at first sight is surprising for an attack. The explanation is the radio model: the Unit Disk Graph Medium is configured with a success ratio of 1.0, and one datagram per node every thirty seconds leaves the channel far from saturation. In these conditions the attacks cannot cause losses, and their cost is paid in energy and stability rather than in delivery. A lossy radio model would produce a different and less favourable picture, and this is noted as a limitation.

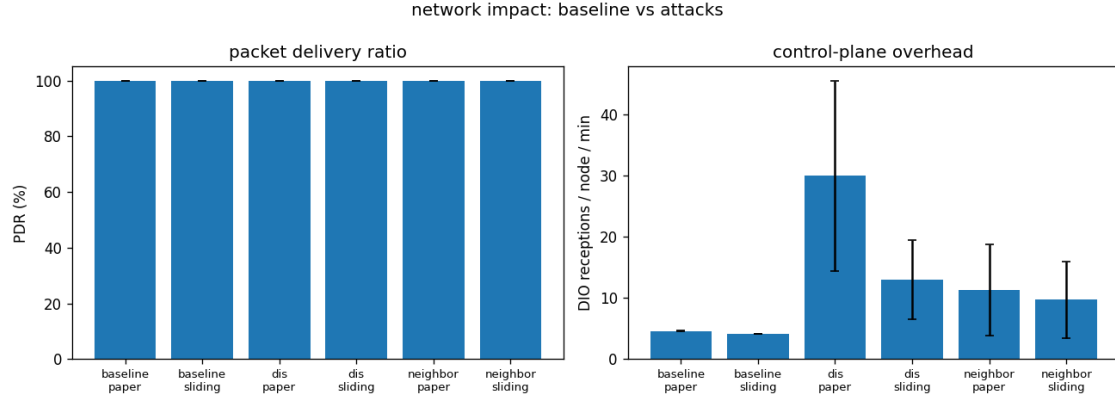


Figure 8: Packet delivery ratio and control-plane overhead, attack-free runs against attacks, for both detector modes

Where the damage does appear is in the control plane, and there it is severe. A single DIS attacker multiplies the number of DIO messages each node receives by six, and 30% of attackers by thirteen, because every solicitation resets the Trickle timer of every node in range. Parent changes rise from 1.00 to 1.44 per node over the run. The neighbour attack is considerably cheaper for the network, since it adds essentially only its own messages. This confirms the asymmetry anticipated in the threat model: the DIS attack is the more efficient of the two from the attacker’s point of view, because it makes honest nodes do the work.

8.7 Sensitivity to the DIS Threshold

The paper’s DIS threshold of 3 was obtained from measurements and its robustness is worth testing. Values of 2, 3 and 5 were evaluated at 20 nodes with one DIS attacker at the paper’s random rate.

DIS threshold	TPR %	FPR %	Latency s
2	100	0.0	225
3 (paper)	100	0.0	225
5	100	0.0	325

Table 5: Sensitivity of the DIS rule to the value of its threshold

Detection is entirely insensitive to the threshold in this range, for the reason already given: honest nodes never exceed 2 messages per window and the attacker produces around 10, so any threshold between the two separates them. Raising the value to 5 only delays detection, when a particular window happens to contain few attacker messages. The paper’s choice sits comfortably in the middle of the usable range.

8.8 Sensitivity to the Attack Rate

The last experiment varies the aggressiveness of the attacker, from one message per second to one every thirty seconds, plus the paper’s random interval.

Attack	Period	Paper mode	Sliding mode
DIS	1 s	100% / 225 s	100% / 5 s
DIS	5 s	100% / 225 s	100% / 25 s
DIS	10 s	100% / 225 s	100% / 45 s
DIS	30 s	100% / 225 s	0% / not detected
DIS	random 5–60 s	100% / 225 s	33% / 575 s
neighbour	1 s	100% / 225 s	100% / 5 s
neighbour	5 s	100% / 225 s	100% / 22 s
neighbour	10 s	100% / 225 s	100% / 28 s
neighbour	30 s	100% / 225 s	100% / 55 s
neighbour	random 5–60 s	100% / 225 s	100% / 38 s

Table 6: Detection rate and latency as a function of attack rate. Cells give True Positive Rate and mean detection latency

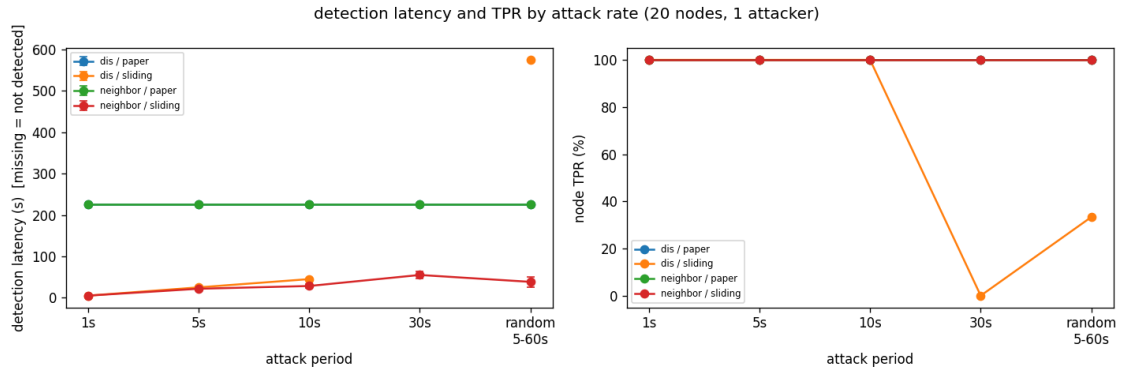


Figure 9: Detection latency and True Positive Rate as a function of the attack period. The sliding mode is faster at every rate but loses the slow DIS attacker entirely

The paper mode detects every rate with the same latency of one window, which is both its strength and its weakness: it is insensitive to how aggressive the attacker is, but it is never faster than 225 seconds. The sliding mode scales with the attack, detecting an aggressive attacker in 5 seconds.

But Table 6 also reveals a **third finding**. The sliding mode fails completely against a slow DIS attacker. A message every 30 seconds places at most two of them inside a 60-second window, and two never exceeds the threshold of 3. That threshold was

calibrated by the authors on a window five times longer, and it does not transfer. The consequence is general: *a detection rule expressed as an absolute count is bound to the length of the window that produced it*, and shortening the window silently weakens it. A sliding implementation must either scale the threshold with the window, which for a 60-second window means a value near 1 and a considerable false-positive risk, or replace the count with a rate. It is worth noting that the neighbour rule suffers nothing of the sort: because it compares a neighbour against its peers rather than against an absolute number, it remains valid at every rate and in both modes.

8.9 Resource Overhead

Component	ROM	RAM
IDS module alone	1988 B	588 B
Stock routing and UDP node, no IDS	44578 B (91%)	7036 B (69%)
Available on the Tmote Sky	49152 B	10240 B

Table 7: Memory footprint measured on the Tmote Sky target

The detector itself is genuinely lightweight, as the paper claims: about 2 kB of ROM and 0.6 kB of RAM, the latter dominated by the neighbour table, with the sliding mode adding roughly 0.5 kB more for the ring buffers. The lookup table for the coefficient k costs 82 bytes.

The complication is elsewhere. Under Contiki-NG v5.2 a stock node running only the routing layer and a UDP application already occupies 91% of the Tmote Sky’s ROM, leaving about 4.5 kB free. The detector fits within that margin, but the instrumented firmware used for these experiments, which prints a detailed record for every message and every decision, does not: it exceeds the available ROM by roughly 10 kB. The measurements were therefore taken on the native simulator target.

This is a genuine finding about the platform rather than about the detector. The paper ran on the same hardware but on Contiki 2.7, whose routing implementation was substantially smaller and left far more room. A production deployment, containing the detector but not the verbose instrumentation, would still fit on the Sky; the logging is an artefact of measurement and not part of the system.

9 Discussion

The reference paper’s method works, and works cheaply, wherever its assumptions hold. The DIS attack is caught perfectly because honest and malicious behaviour do not overlap at all, and the rule that separates them is a single comparison. A lone neighbour attacker is caught whenever at least one monitor outside the blind range hears it. The false positives are neither mysterious nor alarming: they are the arithmetic consequence of applying a formula based on means and standard deviations to counts that are almost always zero or one.

The weaknesses found here are structural rather than accidental. Both the blind spots and the masking effect follow from the same decision, namely computing the dispersion over a set that contains the suspect, and both are invisible in an evaluation that reports only aggregate detection rates. They do not make the method useless, since the distributed design compensates for individual failures, but they do bound what it can promise: neither “100% detection” nor a guarantee against a small coordinated group.

The sliding window behaves as hypothesised on latency and false positives, and better than hypothesised on two counts, since it partly resists masking and substantially reduces the traffic the DIS attack generates. Its two failures are more instructive than its successes. The over-blocking in attack-free runs and the blindness to slow DIS attackers both come from inheriting parameters, the block counter and the DIS threshold, that were tuned for a different window length. The lesson generalises beyond this project: when the observation window of a detector is changed, every parameter expressed in absolute terms must be re-derived, not carried over.

Finally, the network-impact measurements are a reminder that an attack can be entirely successful without affecting the metric one instinctively checks. Packet delivery was perfect in every configuration, including with twelve attackers out of forty; the damage was a thirteen-fold increase in control traffic, which on battery-powered devices translates directly into shortened lifetime.

10 Limitations and Future Work

The evaluation is entirely simulated, with static nodes, a lossless radio model, and a configuration in which ground truth is known exactly. The delivery and delay results in particular should be read as lower bounds on the damage: a lossy channel or heavier application traffic would allow the additional control messages to cause real losses.

The gap between the multi-attacker detection rates measured here and those reported in the paper deserves a dedicated study. Two explanations were identified but not isolated: the placement of attackers relative to one another, and the difference between an attacker that floods at a fixed rate and one that rebroadcasts the messages it receives. Implementing the second variant and repeating the matrix would settle the question.

Three directions follow naturally from the findings. The masking effect could be attenuated by estimating the dispersion robustly, for instance from a median absolute deviation, or by excluding already-suspected neighbours from the profile before recomputing it. The sliding detector needs its response parameters re-derived and its DIS rule expressed as a rate rather than a count. And the whole system should eventually be measured on real hardware, which requires trimming the instrumentation to fit the memory available.

11 Conclusion

The question this project set out to answer was whether a distributed, low-resource anomaly-based IDS can detect RPL neighbour and DIS attacks using only what each node can observe locally. The answer is yes, with qualifications that the reproduction itself made visible.

The DIS attack is detected with a True Positive Rate of 100% and a False Positive Rate of 0% at every network size and every attacker proportion tested, exactly as the reference paper reports. A single neighbour attacker is likewise detected reliably. The detector costs about 2 kB of ROM and 0.6 kB of RAM, needs no coordination between nodes, and adds no traffic of its own.

The qualifications concern the threshold formula. Because it computes the standard deviation over a set that includes the attacker, it cannot detect a lone attacker at certain neighbourhood sizes regardless of how aggressive that attacker is, and it degrades sharply when several attackers share a monitor. Both effects were predicted analytically and then confirmed in simulation.

The comparison between observation windows shows a clear trade-off. A sliding 60-second window reduces detection latency by an order of magnitude and, unexpectedly, both resists attacker masking better and halves the control traffic that the DIS attack provokes. Against this it produces considerably more false positives, over-blocks the network unless the response parameters are re-derived, and fails entirely against a slow DIS attacker because an absolute threshold does not survive being moved to a shorter window.

Table 8 summarises the outcome against the hypotheses formulated before the experiments.

#	Hypothesis	Outcome	Verdict
H1	A neighbour attacker deviates from the DIO profile and is detected	Confirmed for a single attacker; bounded by neighbourhood size and by the number of attackers	Partly
H2	A DIS attacker exceeds the per-window DIS threshold	Confirmed without exception, in every configuration and both modes	Yes
H3	Raising the attacker proportion increases attack traffic but also degrades detection	Confirmed, and more strongly than the paper suggests: 100% to 33% to 5%	Yes
H4	A sliding window lowers latency at the price of more false positives	Confirmed: 225 s to 28 s, false positives 1.5% to 11.1%, plus over-blocking	Yes
H5	The IDS has a measurable but modest resource cost and needs no central monitor	Confirmed: 2 kB ROM, 0.6 kB RAM, no detection traffic; but the platform itself is nearly full	Yes

Table 8: Outcome of the experiments against the initial hypotheses

Overall: the reference model is a good piece of engineering for the problem it addresses, and it reproduces on a modern stack a decade after publication. What this work adds is a clearer view of its boundaries. The blind spots and the masking effect are consequences of one design decision in the threshold, and the sliding-window experiment shows that the parameters of such a detector cannot be separated from the window they were measured on. Both are useful cautions for anyone deploying a threshold-based intrusion detection system on constrained devices.